OSNT Mini-Project Report

Project #16

Implementation of Rate-Limited Sender Functionality in the picoquic Library

CUBIC congestion control for QUIC

Project team

Aryan Bokolia

Affan Arshad

Soumyadeep Raj

Kevin Shiju George

Aryan Palimkar

9 November 2026

Abstract

This project implements Rate-Limited Increase (RLI) as an option for CUBIC in picoquic. RLI uses the largest recorded flight size on each path to cap congestion-window growth. A window reduction starts a new flight-size history. We compared original CUBIC and CUBIC with RLI using Linux network namespaces on Ubuntu. The workloads included a fixed-rate stream, bulk transfers, and competing connections on a shared bottleneck.

With a fixed application rate, RLI grew the window to its slow-start limit while throughput stayed essentially unchanged. The bulk and competing-flow tests measured behavior under higher demand. The live traces confirmed slow-start capping, and focused C tests exercised the selected congestion-avoidance cap. Both builds passed their registered regression suites, and the RLI traces passed the applicable checks.

Contents

Introduction 4–5
Literature Review 6–8
Proposed Approach 9–10
Implementation 11–13
Result and Analysis 14–18
Conclusions and Future Work 19
References 20
Appendix: Run Records 21

All sizes labelled KiB use 1024 bytes. Mbit/s uses 1,000,000 bits per second. Means and sample standard deviations summarize three repetitions.

1. Introduction

1.1 Motivation and transport context

Transport congestion control decides how much outstanding data a sender may place on a network path. Its congestion window, cwnd, is a permission limit rather than a measurement of current demand. An application can generate data slowly even when the path and congestion controller permit more transmission. In that case, a low sending rate does not by itself mean that the network is congested. Treating every quiet interval as evidence for more capacity is also unsafe: unused permission has not actually exercised the path.

Interactive applications, periodic updates, and media-like frame generation provide practical examples. A producer may release a small frame, wait for the next production interval, and repeat. Acknowledgements can arrive throughout this process although the sender never fills the permitted window. If every acknowledged byte triggers unrestricted growth, cwnd can become much larger than the largest flight that was tested. If growth is always suppressed, the sender retains a conservative window even while successfully delivering successive flights. The project studies the space between these two behaviours.

QUIC is an appropriate setting because it separates reliable transport operation from the application workload while allowing the congestion controller to be implemented in a user-space library. RFC 9000 defines the transport, including streams and flow control [4]. The present work changes the sender-side CUBIC controller in picoquic; it does not redesign the application protocol, cryptography, or receiver. A receiver still acknowledges data normally. This keeps the experiment focused on the relationship between transmitted flight size and future sending permission.

1.2 Meaning of rate limitation

Three quantities must be distinguished: application supply, bytes currently in flight, and cwnd. The application can offer less data than cwnd allows. FlightSize represents outstanding congestion-controlled data, while cwnd bounds how much may remain outstanding. Receiver flow control can impose a further limit. Pacing introduces deliberate spacing between packets and should not automatically be mistaken for insufficient application demand. The live workload here deliberately limits application supply; receiver-credit exhaustion is outside its measured coverage.

A larger congestion window therefore need not produce a higher receive rate. If a workload releases only 1200 bytes twenty times per second, its nominal payload production rate is fixed near 192 kbit/s. Raising its window from 15 KiB to 30 KiB cannot create extra application bytes. The expected observation is controlled growth of permission followed by a stable ceiling. This distinction guides both workload design and the interpretation of the final graphs.

The project goal is consequently a change in sender behaviour with observable bounds, rather than a promise of throughput improvement in every application. Its usefulness must be assessed through state accounting, reset behaviour, and compatibility as well as transfer completion. The following sections connect those requirements to the inspected implementation and the retained experiments [11, 12].

1.3 Problem addressed in this implementation

The pre-RLI picoquic CUBIC implementation uses sender-limited timing information to suppress ordinary window increases when the application has not used its allowance. It also relies on common slow-start helpers that can gate increases on whether the sender was congestion-window blocked. Removing only one timing condition would therefore be incomplete. The new behaviour has to permit the candidate increase at the correct locations, then apply a consistent limit before exposing the new cwnd to subsequent transmission [11].

The central implementation problem is to preserve evidence of actual flight-size peaks. An acknowledgement reduces outstanding data, so measuring only at ACK time can miss the largest flight that existed between acknowledgements. The maximum must be updated where send and removal accounting changes bytes_in_transit. A congestion-window reduction begins a new observation interval, so a previously large flight must not silently authorize future growth indefinitely after recovery. These requirements cross the sender, congestion-control, and logging modules.

1.4 Objectives and deliverables

The work has five connected objectives. First, inspect picoquic's congestion-control structures and build a genuine pre-change baseline. Second, construct the supplied Linux namespace topology and collect original CUBIC traces. Third, implement optional RLI state, bounded growth, and reduction resets for CUBIC. Fourth, repeat identical workloads with RLI enabled and retain enough metadata to trace each graph to its source and executable. Fifth, validate the expected growth and record limits that cannot be resolved from the campaign alone.

The Ubuntu experiment records are archived in the repository under `results/`. The main campaign compares fixed-rate and bulk workloads with three repetitions per mode. A second experiment runs two competing connections on a shared bottleneck. The archive contains measurements, plots, and raw traces [12].

2. Literature Review

2.1 Window-based congestion control

RFC 5681 provides the TCP baseline for slow start, congestion avoidance, and recovery [5]. Slow start probes available capacity through comparatively rapid window increases, whereas congestion avoidance grows more cautiously. These mechanisms relate acknowledgement feedback to outstanding transmission. The relevant lesson for this project is that successfully acknowledging a small amount of data is weaker evidence than successfully carrying a full large window. The review therefore emphasizes how much of the permitted window was exercised, rather than the existence of ACKs alone.

A flight-history-based bound makes this distinction explicit. It ties future permission to an outstanding-data quantity that the sender has actually observed. For implementation, this means that data-accounting semantics matter as much as the final growth formula. A maximum computed after acknowledgement removal alone could understate a peak, whereas a maximum kept unchanged through genuine reductions could retain stale permission. Both errors can occur even if the top-level congestion-control equation appears reasonable.

2.2 QUIC transport and congestion feedback

RFC 9000 supplies the QUIC transport context [4]. Independent streams and connection-level operation make it possible for demand and available sending credit to vary without implying a change in path capacity. For the project, the relevant separation is between the application's production schedule and the transport controller's allowance. The experiment changes the latter while keeping the workload definition fixed, so the cause of an observed cwnd difference is easier to identify.

RFC 9002 describes QUIC loss detection and congestion control [3]. Its underutilization guidance discourages congestion-window increases when bytes in flight are below the window for reasons other than pacing. This provides a conservative reference for the original implementation. The new experiment asks whether bounded growth can replace that suppression while retaining the ordinary congestion response. It does not treat RLI as permission to ignore loss, ECN, or the distinction between an ACK event and a congestion event.

2.3 CUBIC as the baseline controller

RFC 9438 specifies the CUBIC window function and its Reno-friendly alternative [2]. The cubic curve is conventionally written W_cubic(t) = C(t - K)^3 + W_max. The controller selects a candidate using both the cubic and Reno-friendly calculations. The RFC also excludes application-limited intervals from elapsed growth time and conservatively suppresses window growth for application-limited flows. These features are directly relevant to the project because enabling bounded growth must not accidentally count idle time as capacity-probing time.

The inspected picoquic fork has a private CUBIC state structure holding phase, threshold, epoch, and curve variables. Its constants include C = 0.4, a loss beta of 3/4, and an ECN beta of 7/8 [11]. These are properties of the tested source; they should not be substituted with values remembered from a different CUBIC implementation. Both experiment versions inherit the fork's controller choices. The measured baseline is therefore the recorded picoquic revision, not a claim that every parameter exactly reproduces every aspect of the RFC.

A further implementation consequence concerns the Reno-friendly shadow window. Applying a limit only to the exposed cwnd can leave an internal candidate above the permitted value. On the next acknowledgement, that state may restore the growth that was just removed. The project clips the shadow window to the allowed result. This is a source-level consistency measure that a plot of cwnd alone would not reveal, and it motivates focused state-transition tests in addition to live transfers.

2.4 Delay-based startup interpretation

RFC 9406 describes HyStart++, which uses delay information to leave slow start and includes protection against premature exit [7]. The inspected picoquic implementation uses its own HyStart-related RTT filter and recovery path. The report does not assert full RFC 9406 equivalence. This literature supports a narrower interpretive point: a startup window reduction need not prove packet loss. In the single-flow runs, recorded retransmissions are zero, so the drop in the bulk cwnd graph is described as consistent with the implementation's delay response.

2.5 Rate-Limited Increase specification

The principal specification is draft-ietf-ccwg-ratelimited-increase-11, published on 6 September 2026 [1]. It introduces maxFS as flight-size history, initializes it using the initial window, updates it when FlightSize changes, and resets it after a window reduction. Growth is then limited according to what one successfully acknowledged maxFS-sized flight would permit. The document gives explicit slow-start and ordinary congestion-avoidance expressions and updates the conservative application-limited guidance for CUBIC and QUIC.

Section 3 defines the limit in terms of the growth allowed after one maxFS-sized flight. Its additive congestion-avoidance example follows RFC 5681. Section 4.5 applies the Section 3 rules to CUBIC [1]. In this implementation, CUBIC computes its usual candidate and the Reno-friendly estimate. RLI caps the larger value at maxFS + send_mtu [11]. This applies the additive example as a conservative implementation choice for the time-based CUBIC curve.

2.6 Implementation and emulation resources

The official picoquic repository provides the implementation context [8]. The local checkout supplies a congestion-control callback interface, per-path transport state, packet accounting, tests, and logging. The project extends those existing facilities instead of introducing a separate transport model. This is why implementation validation must consider packet accounting and internal CUBIC state, rather than merely draw a theoretical congestion curve.

Linux network namespaces isolate networking resources such as interfaces and routing tables [9]. Virtual Ethernet links connect the endpoint and router namespaces. The tc-netem manual describes delay, rate, queue limits, and other emulation controls [10]. The project uses these facilities to execute actual QUIC connections over a controlled local path. Accordingly, the experiments are network emulation, not a discrete-event simulation with synthetic transport results.

3. Proposed Approach

3.1 Design principles

The proposed solution introduces the smallest state needed to connect congestion-window growth to actual flight history. State is stored per path because bytes in transit and CUBIC state are already path specific. The feature is optional so that the original behaviour can be used for compatibility checks. Its activation string is rli=on with CUBIC selected. DCUBIC has a separate initializer that keeps RLI disabled despite sharing controller machinery [11].

Each path records maxFS, the currently computed limit, a cap-active flag, and a saturating reset counter. On initialization, maxFS is set to the source's initial window. After a send or removal changes bytes_in_transit, the accounting helper retains the larger of the existing maximum and the new flight size. On a genuine cwnd reduction, the helper clears maxFS and the cached cap state, and increments the reset counter. Subsequent flight updates rebuild the observation history.

3.2 Candidate growth and limiting rule

The growth candidate remains part of the existing controller. In RLI slow start, the implementation obtains the normal target-window estimate and adds newly acknowledged bytes using saturating addition. This bypasses a legacy helper gate that would otherwise suppress the very application-limited growth under investigation. In congestion avoidance, the controller computes the cubic and Reno-friendly candidates and selects the larger. Epoch adjustment for a sender-limited interval remains in place.

```
L_SS = saturating_multiply_by_two(maxFS)
L_CA = saturating_add(maxFS, send_mtu)
bounded_candidate = min(candidate, L_phase)
cwnd_after_ACK = max(cwnd_before_ACK, bounded_candidate)
```

The final maximum with the previous window is essential: the limiter constrains increases rather than creating a new congestion signal. If an epoch adjustment produces a smaller candidate, ordinary ACK processing preserves the previously permitted window. Real loss, ECN, timeout, or startup congestion responses remain responsible for reductions and observation resets. The implementation then clips W_reno to avoid leaving an internal window estimate above the exposed result.

3.3 Controlled topology

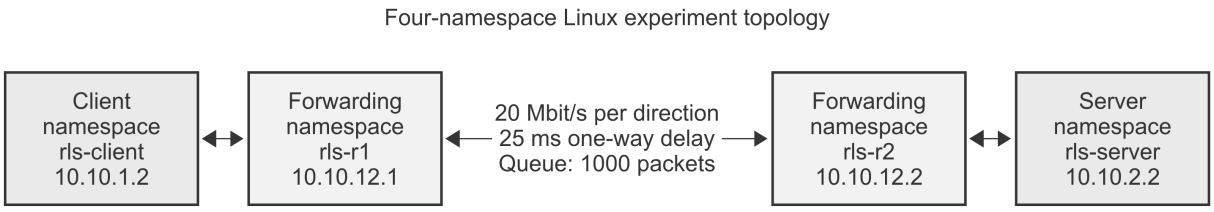


Figure 1: Four-namespace Linux experimental topology. The server sends the measured payload to the client.

The main topology has two endpoint namespaces and two forwarding namespaces. The core link carries 20 Mbit/s in each direction with 25 ms of one-way delay and a 1000-packet queue limit. The base round-trip time is approximately 50 ms. The supplementary dumbbell connects two client/server pairs across the same shaped link. Each supplementary run starts both transfers together [11, 12].

3.4 Traceability and analysis

Each run records the command, binary identity, configuration, performance data, and qlog traces. The source revision for the actual baseline binary is recorded in its `provenance.txt`. The sender qlog provides the window and flight-size samples. The server connection CSV supplies byte counts and connection times. The analysis weights each window sample by the time until the next sample. The comparison reports the means and sample variability across the three repetitions.

4. Implementation

4.1 Structural design and responsibilities

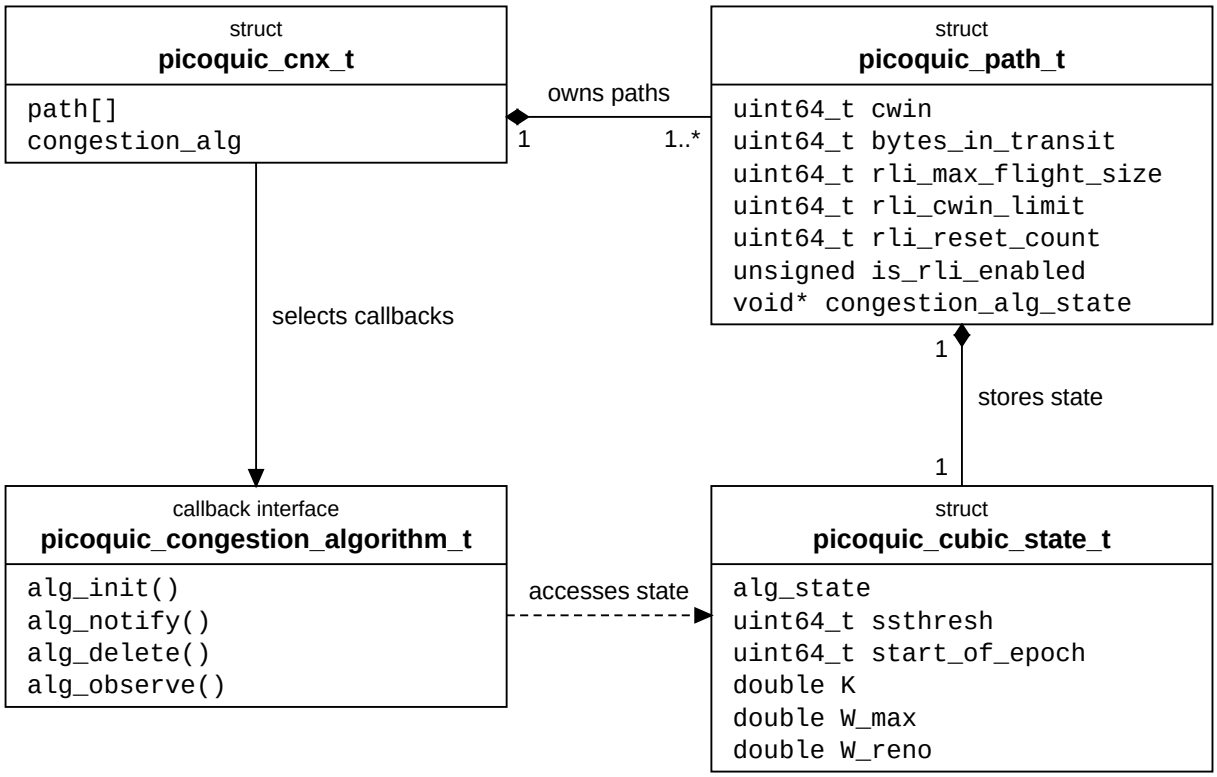


Figure 2: Class-style structural diagram of the core C structs and callback interface. Selected fields are shown.

picoquic_cnx_t holds path references and the selected congestion-control interface. A path holds cwnd, flight accounting, RLI fields, and an opaque pointer to its algorithm state. picoquic_cubic_state_t remains the private CUBIC state. ACK information is passed through picoquic_per_ack_state_t to the notification callback. The diagram uses C struct and callback-interface stereotypes because the implementation is C; it does not invent C++ classes or member methods [11].

The sender module updates flight history where packet enqueue and dequeue accounting occurs. CUBIC computes limits, applies growth, and resets observation state during recovery. The logging modules export the resulting values. This distribution follows existing ownership: packet accounting remains in sender.c, policy remains in cubic.c, and diagnostics remain in the log writers and converters.

State is per path even though the live campaign selects one path. Focused tests create independent path instances to check isolation. That unit-level check does not establish correct live multipath behaviour, which would require dedicated connections and trace selection.

4.2 Initialization and packet accounting pseudocode

```
ON_CUBIC_INITIALIZE(path, option):
    path.rli_enabled = option in {"rli", "rli=on", "on"}
    initialize_existing_CUBIC_state(path)
    path.cwnd = INITIAL_CWND
    path.maxFS = INITIAL_CWND if enabled else 0
    path.limit = 2 * INITIAL_CWND if enabled else 0
    path.reset_count = 0
    path.capped = false

ON_FLIGHTSIZE_UPDATE(path):
    if path.rli_enabled:
        path.maxFS = max(path.maxFS, path.bytes_in_transit)

ON_PACKET_ENQUEUE(path, packet_bytes):
    perform_existing_enqueue_and_flight_accounting()
    ON_FLIGHTSIZE_UPDATE(path)

ON_PACKET_DEQUEUE(path, packet):
    perform_existing_removal_and_flight_accounting()
    ON_FLIGHTSIZE_UPDATE(path)
```

INITIAL_CWND is 10 × PICOQUIC_MAX_PACKET_SIZE in this source, or 15,360 bytes in the recorded runs. Initializing maxFS to that value explains why a flight smaller than 15 KiB can still authorize growth toward 30 KiB in slow start. The maximum is not redefined as merely the largest small application frame. The source's send_mtu is used for the additive congestion-avoidance limit.

4.3 Implementation map

Source file / component	**Implemented responsibility**
picoquic_internal.h	Per-path RLI fields, flags, and reset counter.
sender.c	Maximum-flight accounting after enqueue and dequeue.
cubic.c	Activation, candidate growth, caps, epoch guard, recovery reset, and DCUBIC isolation.
logwriter.c; loglib/qlog*.c	Binary/direct qlog fields and optional-field conversion.
plot_cwnd.py; experiments/*.py	Time-unit handling, trace parsing, graphs, validation, and campaign summaries.

Cap arithmetic saturates at UINT64_MAX. Reset counting also saturates, avoiding wraparound in a long-running path. The Ubuntu verification did not separately exercise the overflow boundary.

4.4 Growth and reduction pseudocode

```
ON_ACK_GROWTH(path, ack, phase):
    old = path.cwnd
    if not path.rli_enabled:
        run_existing_CUBIC_ACK_logic(); return
    candidate = existing_phase_candidate_with_RLI_growth(ack)
    limit = SAT_ADD(path.maxFS, path.send_mtu) if phase == CA
            else SAT_DOUBLE(path.maxFS)
    candidate = max(candidate, old)       # ACK cannot shrink cwnd
    path.capped = candidate > old and candidate > limit
    path.cwnd = max(old, min(candidate, limit))
    path.W_reno = min(path.W_reno, path.cwnd)
    path.limit = limit

ON_REAL_WINDOW_REDUCTION(path, congestion_event):
    run_existing_CUBIC_recovery_response(congestion_event)
    if path.rli_enabled:
        path.maxFS = 0; path.limit = 0; path.capped = false
        path.reset_count = SAT_ADD(path.reset_count, 1)
```

The pseudocode summarizes the RLI logic and does not replace the existing ACK, recovery, or RTT state machine. The first live pilot exposed an ACK-driven decrease after an epoch adjustment. The corrected helper preserves the old window in that path. Genuine reductions still use recovery and clear maxFS. Spurious recovery may restore CUBIC window and epoch state, but does not restore the pre-reduction flight maximum [11].

4.5 Verification and instrumentation

qlog exports rli_max_flight_size, rli_cwnd_limit, rli_capped, and rli_reset_count. The counter makes a reset observable when the flight has already rebuilt before the next snapshot. The trace validator rejects unreset decreases, uncapped growth, missing cap exercise, and maximum-flight values below observed flight. A retained peak above both adjacent samples is accepted as a possible intermediate event and counted, rather than incorrectly rejected.

Saved verification	**Outcome and scope**
Full regression	Both registered CTest entries passed for both versions. The baseline ran 667 internal checks and the RLI build ran 668.
Focused C checks	The `cubic_rli` test covered growth caps, packet accounting, loss, ECN, timeout, spurious recovery, and CUBIC epoch handling.
Live trace checks	All twelve RLI connections passed the applicable observed-invariant checks.

5. Result and Analysis

5.1 Experimental parameters

The campaign ran on Ubuntu 26.04 LTS with the `7.0.0-38-generic` x86-64 kernel. The environment record identifies GCC 15.2.0 and CMake 4.2.3 [12]. Each revision ran the same scenarios on Linux network namespaces. Netem used 20 Mbit/s, 25 ms one-way delay, and a 1000-packet limit in each direction. The nominal bandwidth-delay product at a 50 ms RTT is about 122 KiB.

The analysis uses the server's application bytes and connection duration for single-flow throughput. It reads the final smoothed RTT, retransmissions, and window samples from the sender qlogs. The graphs below use the archived Ubuntu figures. Detailed traces show repetition one. The tables use all three repetitions in each mode.

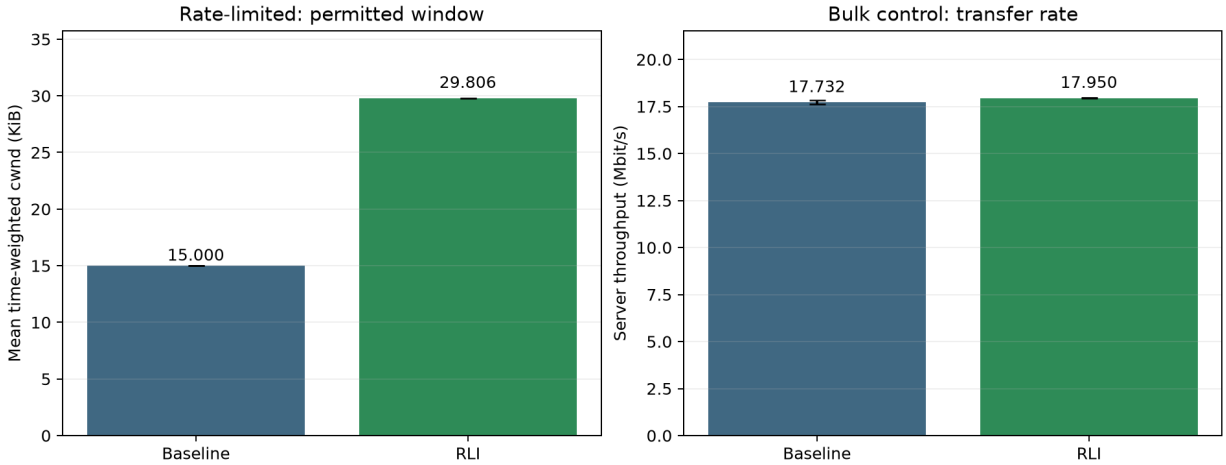


Figure 3: Three-run means with sample standard-deviation bars for fixed-rate cwnd and bulk throughput.

5.2 Fixed-rate workload

Mode	Rate (Mbit/s)	Final cwnd (KiB)	Mean cwnd (KiB)	Final RTT (ms)
Original	0.190179	15.00	15.00	51.115
RLI	0.190186	30.00	29.81	50.944

Each transfer delivers 480,000 bytes. The application supplies approximately 192 kbit/s, so both versions measure about 0.190 Mbit/s. The largest observed flight is 3426 bytes. The initialized 15 KiB maxFS therefore sets the 30 KiB slow-start ceiling. This increase gives the sender additional capacity for a later burst. The experiment does not measure that burst.

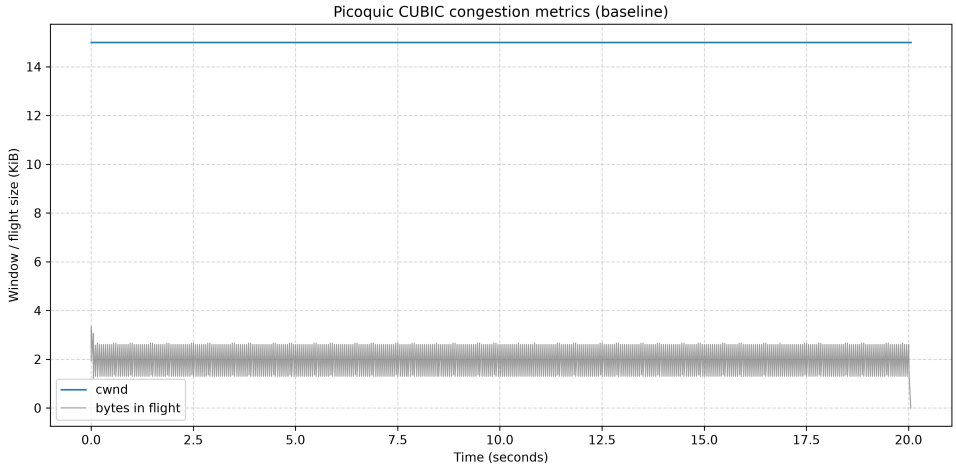


Figure 4: Original CUBIC, fixed-rate repetition one.

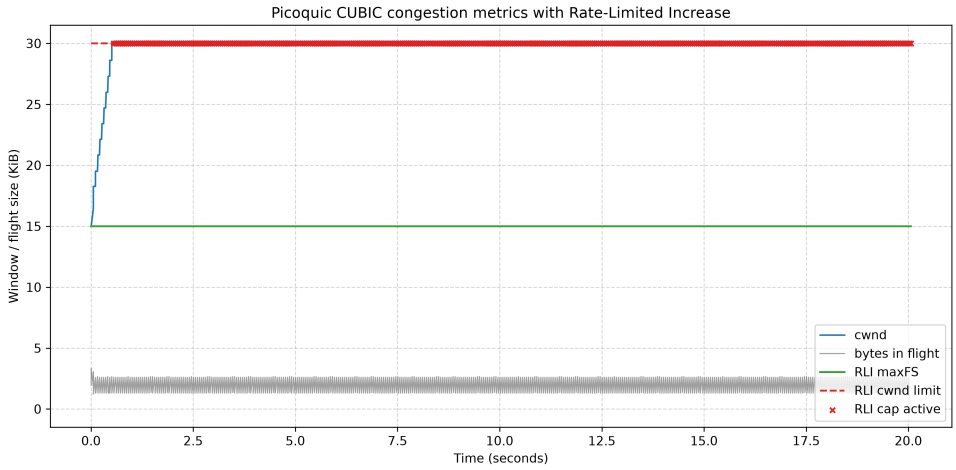


Figure 5: CUBIC with RLI, fixed-rate repetition one.

5.3 Single-flow bulk transfers

Mode	Rate (Mbit/s)	Final cwnd (KiB)	Mean cwnd (KiB)	Final RTT (ms)
Original	17.731656	294.59	213.42	109.409
RLI	17.949780	264.92	234.48	119.322

Mean throughput rises by 1.23% in this campaign. The paired differences are 0.208, 0.135, 0.312 Mbit/s. These three repetitions describe this test path. Final smoothed RTT is an endpoint statistic. A queue trace would be needed to attribute delay changes to occupancy at the bottleneck. The twelve single-flow runs record zero retransmissions.

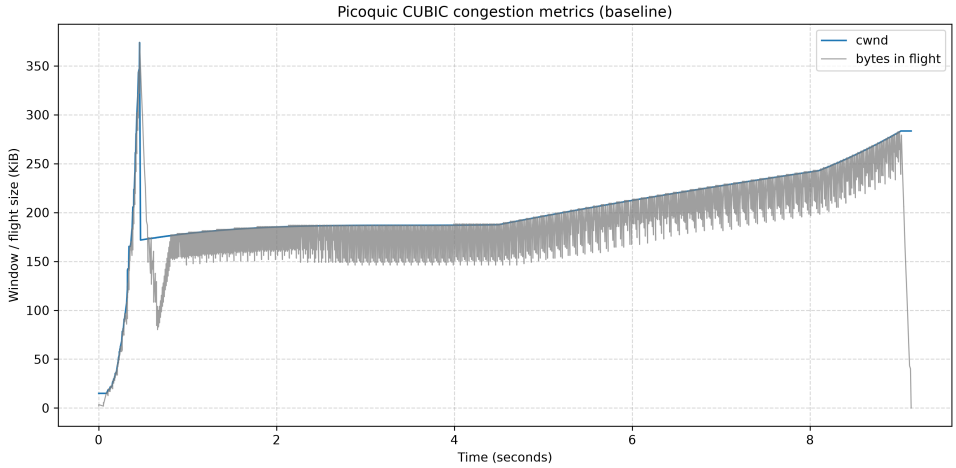


Figure 6: Original CUBIC, bulk repetition one.

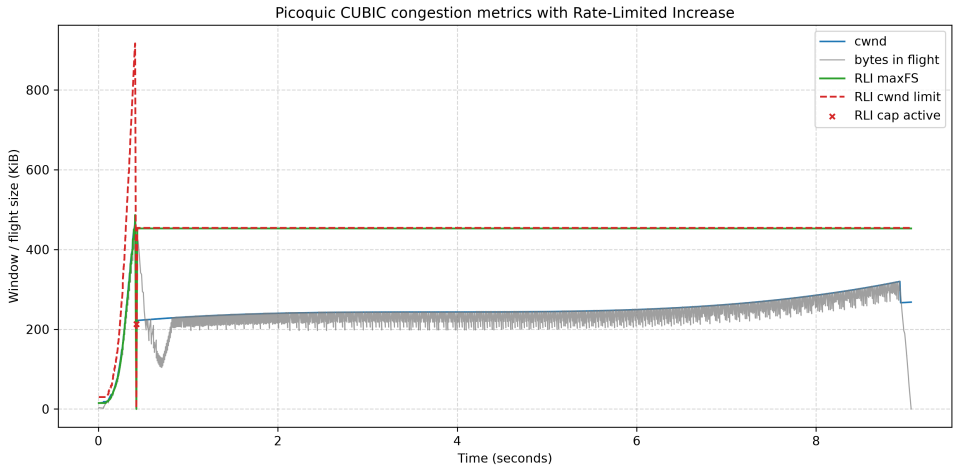


Figure 7: CUBIC with RLI, bulk repetition one.

5.4 Two flows sharing the dumbbell

Mode	Combined Mbit/s	Jain index	Mean retransmissions
Original	18.665	0.99953	450.0
RLI	18.625	0.99984	432.7

Combined throughput uses the interval from the launch of the clients to the exit of both clients. The Jain index uses each flow's rate over its own connection interval. The index is close to one in both modes. Both connections use the same algorithm in a given experiment. The retransmission totals show recovery activity in these runs.

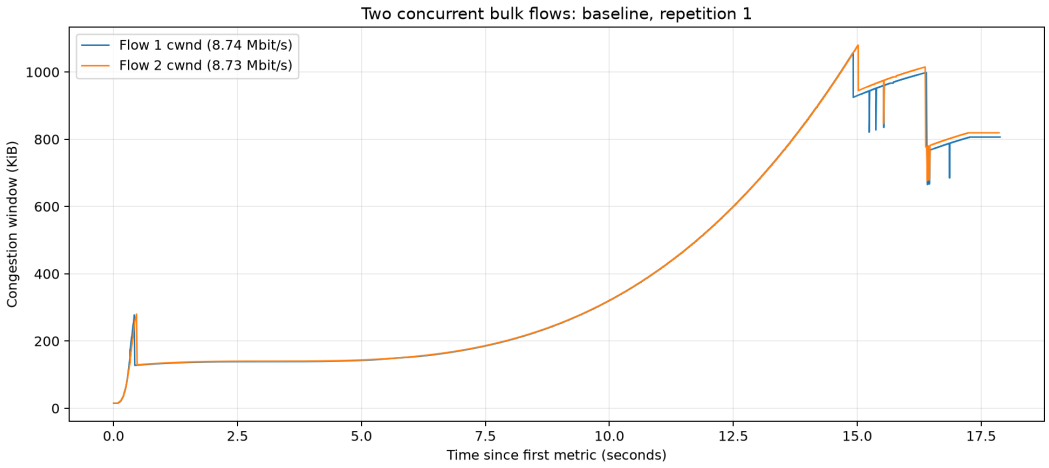


Figure 8: Two competing flows with original CUBIC, repetition one.

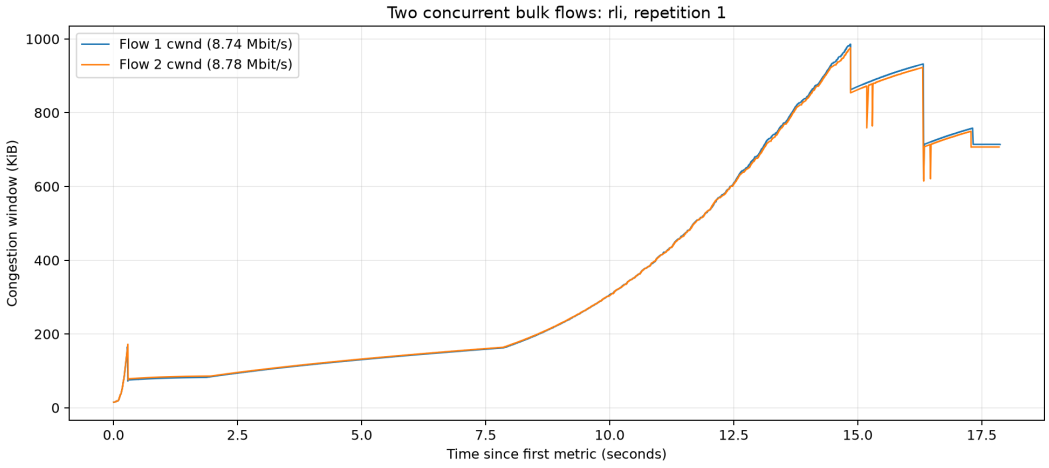


Figure 9: Two competing flows with RLI, repetition one.

5.5 Validation and comparison

All six fixed-rate server qlogs stay in slow start. Each of the six bulk qlogs records a transition to congestion avoidance. The RLI trace checks pass for all six single-flow RLI runs and all six competing RLI connections. The fixed-rate cases show an active cap. The RLI bulk cases record a window reduction and a flight-history reset.

Aspect	Reference behavior	Result in picoquic
Application limitation	RFC 9438 suppresses growth in application-limited intervals [2].	RLI grows cwnd from 15 to 30 KiB at the slow-start cap.
Growth bound	The RLI draft bounds increases using flight history [1].	The live fixed-rate test reaches the slow-start cap. The focused test exercises the selected congestion-avoidance cap.
Window retention	RFC 7661 considers the lifetime of a window that has not been exercised [6].	The implementation covers bounded growth. The experiments do not evaluate window aging.

5.6 Test coverage

A live test with a bulk phase followed by a fixed-rate phase on the same connection would exercise RLI under rate limitation in congestion avoidance. A final burst would measure the effect of the larger window on completion time. The competing-flow experiments compare two instances of the same algorithm. Controlled packet loss, competition against other algorithms, receiver flow-control limits, and multipath behavior remain separate test cases.

6. Conclusions and Future Work

This project implements bounded growth for a rate-limited CUBIC sender in picoquic. The sender tracks the maximum flight size per path and clears that history when the congestion window falls. Both builds passed their registered regression suites.

The Ubuntu experiments show that RLI raises the fixed-rate congestion window from 15 to 30 KiB, as expected from the slow-start rule. The two versions deliver the same amount of data at about 0.190 Mbit/s. Bulk and competing-flow measurements describe behavior when the application supplies more data. All twelve RLI connections pass the applicable trace checks.

A later experiment should bring the sender into congestion avoidance, reduce the offered rate, and then release a burst. This would test the congestion-avoidance cap in a live limited transfer and measure burst completion time. Controlled loss and tests with different competing algorithms would extend the recovery and fairness evaluation.

References

[1] M. Welzl, T. Henderson, G. Fairhurst, and M. P. Tahiliani. Increase of the Congestion Window when the Sender Is Rate-Limited. Internet-Draft, draft-ietf-ccwg-ratelimited-increase-11, 6 September 2026. Sections 3 and 4.5. https://www.ietf.org/archive/id/draft-ietf-ccwg-ratelimited-increase-11.html

[2] L. Xu, S. Ha, I. Rhee, V. Goel, and L. Eggert. CUBIC for Fast and Long-Distance Networks. RFC 9438, August 2023. Sections 4.2 and 5.8. https://www.rfc-editor.org/rfc/rfc9438.html

[3] J. Iyengar and I. Swett. QUIC Loss Detection and Congestion Control. RFC 9002, May 2021. Section 7.8. https://www.rfc-editor.org/rfc/rfc9002.html

[4] J. Iyengar and M. Thomson. QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000, May 2021. https://www.rfc-editor.org/rfc/rfc9000.html

[5] M. Allman, V. Paxson, and E. Blanton. TCP Congestion Control. RFC 5681, September 2009. https://www.rfc-editor.org/rfc/rfc5681.html

[6] G. Fairhurst, A. Sathiaseelan, and R. Secchi. Updating TCP to Support Rate-Limited Traffic. RFC 7661, October 2015. Experimental. https://www.rfc-editor.org/rfc/rfc7661.html

[7] P. Balasubramanian, Y. Huang, and M. Olson. HyStart++: Modified Slow Start for TCP. RFC 9406, May 2023. https://www.rfc-editor.org/rfc/rfc9406.html

[8] Private Octopus. picoquic: official source repository. The local experiment revisions are identified separately in Appendix A. https://github.com/private-octopus/picoquic

[9] Linux man-pages project. network_namespaces(7): overview of Linux network namespaces. https://man7.org/linux/man-pages/man7/network_namespaces.7.html

[10] iproute2 manual. tc-netem(8): Network Emulator. https://man7.org/linux/man-pages/man8/tc-netem.8.html

[11] Local project sources and implementation records: picoquic/cubic.c, sender.c, picoquic_internal.h; picoquictest/congestion_test.c; PROJECT_PROGRESS.md, IMPLEMENTATION_PLAN.md, and results/README.md; topology/setup.sh; experiments/collect_results.py and validate_rli.py.

[12] OSNT Project, Ubuntu experiment evidence, repository revision `33455bd1`. See the current `results/` bundle for the measured results, figures, source identifiers, and raw qlogs. Experiment archive.

Appendix A. Run records

Workload	Mode	Run	Mbit/s	Duration (s)	Final cwnd (KiB)
rate-limited	baseline	1	0.190202	20.189041	15.00
rate-limited	rli	1	0.190171	20.192343	30.00
rate-limited	baseline	2	0.190196	20.189743	15.00
rate-limited	rli	2	0.190194	20.189867	30.00
rate-limited	baseline	3	0.19014	20.195616	15.00
rate-limited	rli	3	0.190192	20.19008	30.00
bulk	baseline	1	17.731435	9.461879	283.41
bulk	rli	1	17.939509	9.352134	268.04
bulk	baseline	2	17.823301	9.41311	304.68
bulk	rli	2	17.957932	9.34254	261.98
bulk	baseline	3	17.640233	9.510798	295.67
bulk	rli	3	17.951898	9.34568	264.75

A.1 Source and trace records

The original build uses commit `5a4eba47`. The RLI build uses commit `60323926`. The Ubuntu archive records executable hashes and contains the original qlogs and test logs. Use each run's provenance record for the baseline executable's source revision. The supplementary dumbbell measurements and topology checks are in the archive's `dumbbell/` directory [12].